SW Engineering CSC648/848 Section 6

Nautilus

AI-assisted PRD drafting

Milestone 2

Architecture & UI Mockups · Vertical SW Prototype

Team 5 – LeoPRD

October 20, 2026

Team Member	Role
Om Pandya	Team Lead, AI Master
Aksh Patel	Frontend Lead, GitHub Co-Lead
Giovannie Silva	AI Master
Leman Yuksel	Scrum Master
Marco Garcia	Backend Lead
Philip Chen	GitHub Master
Vineela Vandanapu	Backend Lead, Product Support

History Table

Version	Date	Description
M1 v1	09/29/2026	Milestone 1 submitted for review
M2 v1	10/20/2026	Milestone 2 submitted for review

Table of Contents

1. Data Definitions V2
2. Functional Requirements V2
3. UI Mockups and UX Flows
4. High-Level Architecture and Database Organization
5. High-Level UML Diagrams
6. Actual Key Risks
7. Project Management

1. Data Definitions V2

Owner: Vineela, Marco, Om **Support:** Leman (review)

All names below are the exact names used in the DB, API, classes and code (snake_case).

Timestamps: editable tables (project, prd, profile) have created_at and updated_at, set automatically by the database. Append-only tables (never edited after insert) carry a single timestamp: created_at, or a more specific one such as answered_at, uploaded_at, retrieved_at or decided_at.

1.1 Data Categories

Category	Items
Raw data	intake_answer, attachment
Main data	project, prd (and its document content)
Supporting data	source, prd_context, profile
Metadata	prd_version, generation_record, timestamps
Not stored	session chat buffer (see 1.10)

1.2 Users and Privileges

profile (linked to the Supabase Auth user):

Field	Type / limits
id	UUID, same as auth user
full_name	max 100
email	unique, valid format
role	pm | engineer | admin
team	max 100
created_at, updated_at	timestamp

Action	PM	Engineer	Admin
Create project / PRD	Yes	No	Yes
Run AI generation and use AI chat	Yes	No	Yes
Edit or delete own project / PRD	Yes	No	Yes
Edit or delete others' projects / PRDs	No	No	Yes

Action	PM	Engineer	Admin
View PRDs	own + shared	assigned/shared	all
Manage users	No	No	Yes

1.3 Project (Folder)

Field	Type / limits
id	UUID
name	max 200
owner_id	FK → profile.id
created_at, updated_at	timestamp

A project is only a container for PRDs and has no AI context of its own.

Deletion rule: deleting a project permanently deletes every PRD in it and all related records. The UI must show a confirmation stating how many PRDs will be deleted.

1.4 PRD

Field	Type / limits
id	UUID
title	text, max 200
owner_id	FK → profile.id
project_id	FK → project.id, nullable (null = standalone PRD), ON DELETE CASCADE
status	draft | in_review | final | assigned
current_version_id	FK → prd_version.id
content_json	JSONB, Tiptap/ProseMirror document, max ~1 MB
content_text	text, plain-text copy for search and AI input (derived)
ydoc_state	nullable. CRDT binary (a Yjs full-state update) holding one Y.XmlFragment named default. Stored as BYTEA, or base64 text if written through the Supabase Python client
doc_schema_version	integer, version of the editor schema (nodes and marks) the document was saved with
revision	integer, incremented on every save; a save sent with a stale revision is rejected (409)
created_at, updated_at	timestamp

When present, ydoc_state is canonical for editing, and content_json and content_text are derived from it by the browser. Seeded or imported PRDs may have only content_json, and the editor builds the Yjs state the first time the PRD is opened.

Saving: Planned for a later milestone, when the Tiptap editor is added. The browser sends ydoc_state, content_json and content_text together. The backend validates content_json against the allowed nodes and marks (1.5), stores all three, and does not decode ydoc_state. Phase 1 has one editor at a time per PRD, and the editor is locked while an AI edit is running. Live multi-user sync is planned for a later phase as a separate Node service.

1.5 PRD Document Format

- **Section types** (section node with a type attribute): problem, goals, non_goals, target_users, user_stories, requirements, success_metrics, risks, open_questions.
- **Allowed nodes:** paragraph, heading (levels 1 to 3), bullet list, numbered list, list item, table (row, cell, header), blockquote, section.
- **Allowed marks:** bold, italic, underline, link (href), ai_generated (attribute: generation_record_id), citation (attribute: source_id).

Any other node or mark is rejected, including AI output. The schema is defined once in shared editor configuration, and doc_schema_version records which version a document was saved with.

1.6 Intake (New PRD Wizard)

The questions are fixed and defined in application code, not in the DB. Each has a stable question_key. The wording and count below are placeholders to adjust with the team.

Order	question_key	Question topic	Required
1	problem	What problem are you solving?	yes
2	target_users	Who are the users?	yes
3	goals	What are the goals?	yes
4	scope	What is in and out of scope?	no
5	constraints	Technical, timeline or business constraints?	no
6	success_metrics	How will success be measured?	no
7	notes	Anything else (research, links, context)?	no

intake_answer

Field	Type / limits
id	UUID
prd_id	FK, cascade
question_key	text
answer_text	max 5,000 chars

Field	Type / limits
skipped	boolean
answered_at	timestamp

Each answer is saved as soon as it is submitted. Unique on (prd_id, question_key).

attachment

Field	Type / limits
id	UUID
prd_id	FK, cascade
file_name	text
file_type	.txt, .md, .pdf, .docx
size_bytes	max 5 MB each, up to 5 per PRD
storage_path	text
uploaded_at	timestamp

Files live in Supabase Storage.

1.7 AI Context

prd_context holds the AI's only long-term memory, one per PRD. Gemini holds no conversation between calls, so each call is built from this record, the current content_text, the session buffer and the user's new message.

Field	Type / limits
id	UUID
prd_id	FK → prd.id, unique, cascade
context_json	JSONB: decisions (list), constraints (list), open_questions (list), source_ids (list), change_log (list of short entries)
running_summary	text, max ~4,000 chars, AI-maintained summary of the PRD's current state
version	integer, incremented on every update
updated_at, updated_by	timestamp, ai | pm

Intake answers stay verbatim in intake_answer, and only running_summary gets compacted. After each AI action the AI returns a structured patch (new decisions, constraints, open questions, updated summary), and the backend validates and merges it. Anything the PM says in chat that should outlive the session must be captured this way.

1.8 Supporting Data

source

Field	Type / limits
id	UUID
prd_id	FK, cascade
url	max 2,048
title	max 300
publisher	max 200
excerpt	max 500
retrieved_at	timestamp
verification_status	unverified | verified | failed
added_by	ai | pm

1.9 Metadata

prd_version

Field	Type / limits
id	UUID
prd_id	FK, cascade
content_json	JSONB
doc_schema_version	integer
created_by	profile id, null if AI
origin	ai_generated | pm_edit | checkpoint
created_at	timestamp

A snapshot is taken before every AI edit (the browser saves first, so the snapshot matches what the PM sees). Undo restores a version by loading its content_json back into the editor.

generation_record

Field	Type / limits
id	UUID
prd_id	FK, cascade
trigger	initial_generation | chat_request
model	e.g. Gemini model name
prompt_version	text
context_version	which prd_context.version was used

Field	Type / limits
grounding_used	boolean
status	success | failed
error_message	text
user_request	optional text, max ~2,000 chars
created_at	timestamp

user_request is an audit and debugging log only. It is never shown in the chat or sent to the AI as conversation memory, and it can be dropped if the team prefers to store nothing about requests.

1.10 Session Chat Buffer (Not Stored)

Chat messages are not saved in the database. The frontend keeps the messages of the current editing session in browser memory.

Property	Definition
Item	message: role (user | assistant), text
Cap	last 10 exchanges (one PM message + one AI reply = 1 exchange), i.e. up to 20 messages, sent with each request
Config	single constant, e.g. CHAT_BUFFER_MAX_EXCHANGES = 10
Lifetime	cleared when the PM leaves the PRD or closes the tab
UI	a divider marks messages older than the cap: “Earlier messages are no longer visible to the AI”
Empty-state text	“This chat resets when you leave. The AI remembers your PRD's saved decisions and summary.”

Each AI chat call is built from: prd_context + current document text + the buffer + the new message.

1.11 Reserved for Later Phases (Priority 3)

classification_suggestion (Phase 2): id, prd_id (cascade), suggested_team, suggested_priority, dependencies (list), required_approvals (list), confidence, status (pending | accepted | rejected), decided_by, decided_at, created_at.

comment (Phase 3): id, prd_id (cascade), author_id, body, created_at.

1.12 Deletion Cascade Summary

Deleting a project deletes its prd rows, which in turn delete intake_answer, attachment, prd_context, source, prd_version, generation_record and the reserved tables. Supabase Storage files don't cascade, so the backend must delete the matching attachment files separately.

2. Functional Requirements V2

Owner: Om, Leman **Support:** Entire team reviews priorities

Priority key: 1 = must have · 2 = desired · 3 = opportunistic

These six M2 workflows retain their parent requirement IDs from M1. Upload PRD is newly introduced as R_12.

2.1 Login

Parent requirement	R_1 – User Authentication
Priority	1
User story	US5

ID	Requirement	Priority
R_1.1	The system shall allow a registered user to log in using valid credentials.	1
R_1.2	The system shall reject invalid login attempts and display an understandable error message.	1
R_1.3	The system shall allow an authenticated user to log out and end the current session.	1
R_1.4	The system shall allow a new user to create an account with an email and password, confirming the password before submitting.	1
R_1.5	The system shall allow a user to sign up or sign in with a Google account.	2

2.2 Create PRD

Parent requirement	R_2 / R_3 / R_4 – Create Project, Guided PRD Input, Generate PRD
Priority	1
User story	US1, US6, US2

ID	Requirement	Priority
R_2.1	The system shall allow a user to create a new PRD project.	1
R_3.1	The system shall collect product information through a guided input workflow.	1
R_3.2	The guided input shall collect information about the problem, target users, goals, features, constraints, and success metrics.	1
R_4.1	The system shall generate a structured PRD draft from the information provided by the user.	1
R_4.2	The generated PRD shall be associated with the user's project and be available for later editing and saving.	1

2.3 Edit PRD

Parent requirement	R_5 – Edit PRD
Priority	1
User story	US3

ID	Requirement	Priority
R_5.1	The system shall allow the owner of a PRD to edit its existing content.	1
R_5.2	The system shall allow the user to save changes made to the PRD.	1
R_5.3	The system shall preserve the most recently saved PRD content when the user leaves and later reopens the document.	1

2.4 Delete PRD

Parent requirement	R_8 – Delete PRD
Priority	1
User story	US5

ID	Requirement	Priority
R_8.1	The system shall allow the owner of a PRD to delete a PRD they no longer need.	1
R_8.2	The system shall ask the user to confirm the deletion before permanently removing the PRD.	1
R_8.3	After deletion, the PRD shall no longer appear in the user's saved PRD list.	1

2.5 Export PRD

Parent requirement	R_11 – Export PRD
Priority	2
User story	US12

ID	Requirement	Priority
R_11.1	The system shall allow the user to export a saved PRD as a file that can be read without an account.	2
R_11.2	The exported file shall contain the current saved version of the PRD.	2
R_11.3	The system shall allow the user to initiate the export from the PRD view or dashboard.	2

2.6 Upload PRD

Parent requirement	R_12 – Upload Existing PRD (new in M2)
Priority	2
User story	New M2 flow; user-story ID TBD

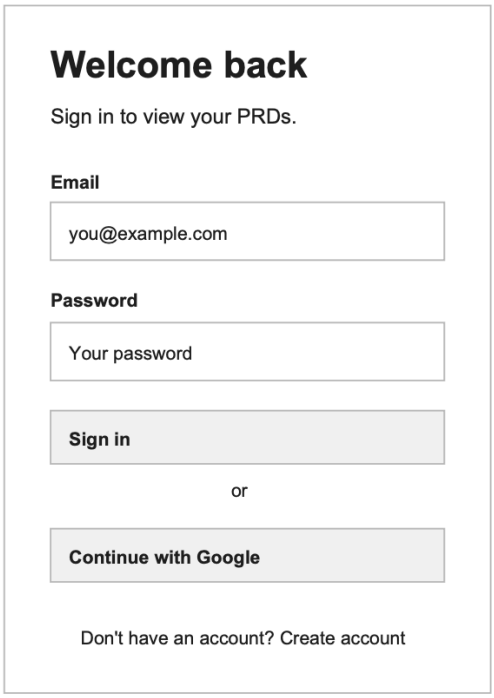
ID	Requirement	Priority
R_12.1	The system shall allow a user to upload an existing PRD file into the application.	2
R_12.2	The system shall validate that the uploaded file is in a supported format before processing it.	2
R_12.3	The system shall create an editable PRD record from the successfully uploaded document.	2
R_12.4	If the upload fails or the file type is unsupported, the system shall display an understandable error message.	2

3. UI Mockups and UX Flows

Owner: Aksh (lead), Leman **Support:** Om (product review)

3.1 Sign In

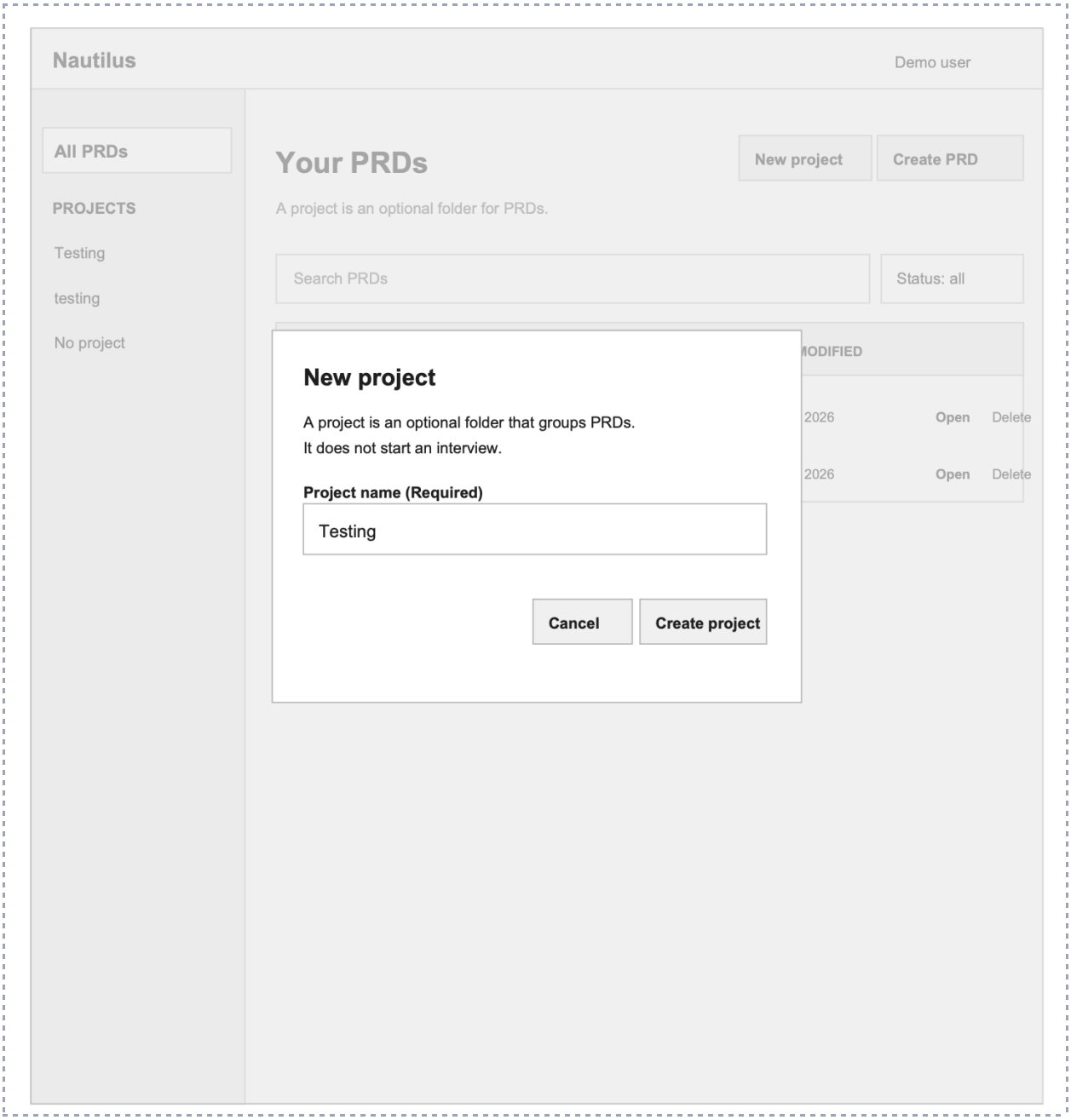
Screens	Login → Dashboard
User story	US5
Requirements	R_1
Description	The user signs in using the Email and Password fields or Continue with Google. After authentication, the dashboard displays the users saved PRDs, projects, and available actions.



3.2 Create Project

Screens	Dashboard → Create PRD
User story	US1
Requirements	R_2
Description	From the dashboard, the user selects New project, enters the required project name, and selects Create project. The project

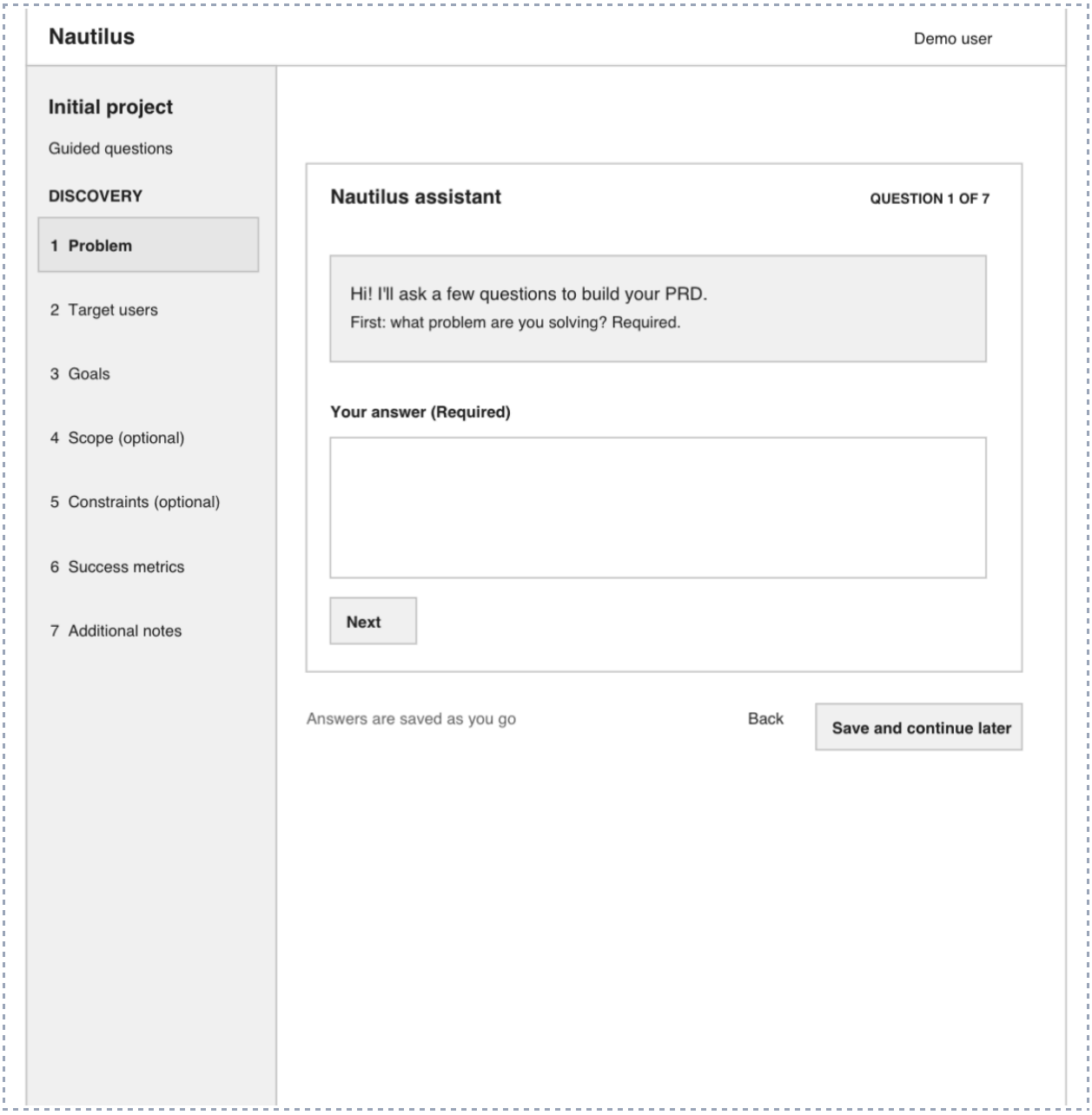
	serves as an optional folder for PRDs.



3.3 Guided PRD Input

Screens	Problem → Target users → Goals → Scope → Constraints → Success metrics → Notes → Review
User story	US6, US9

Requirements	R_3, R_9
Description	The user answers the guided interview questions, using Next to advance and Save and continue later to pause. A review screen lists the seven answer topics and provides an Edit action for each.



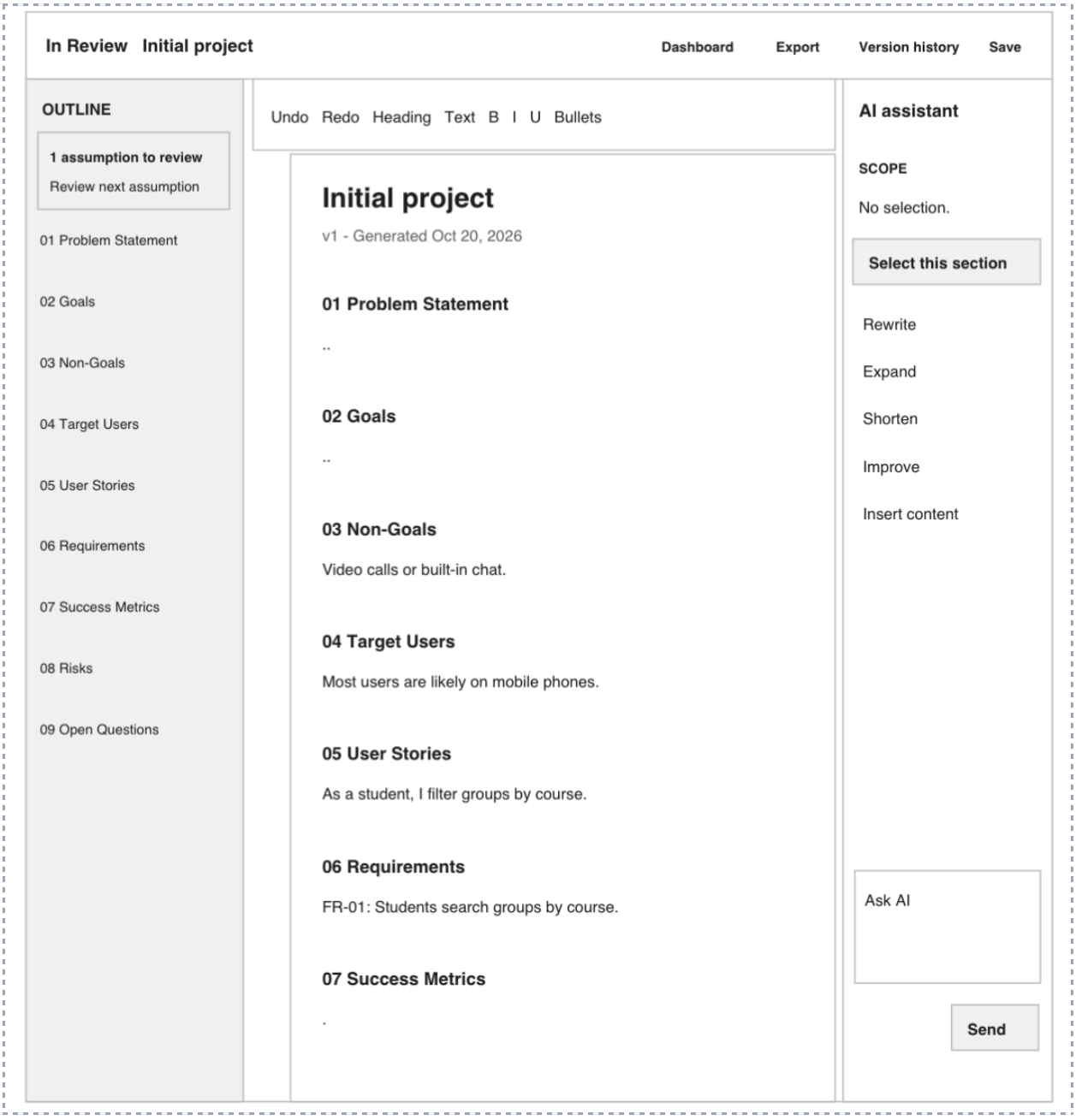
3.4 Generate PRD

Screens	Review Inputs → Generate → Loading → PRD Editor
User story	US2
Requirements	R_4
Description	After reviewing the answers, the user selects Generate PRD and sees a ready confirmation. Selecting Open PRD displays the structured document and its outline in the editor



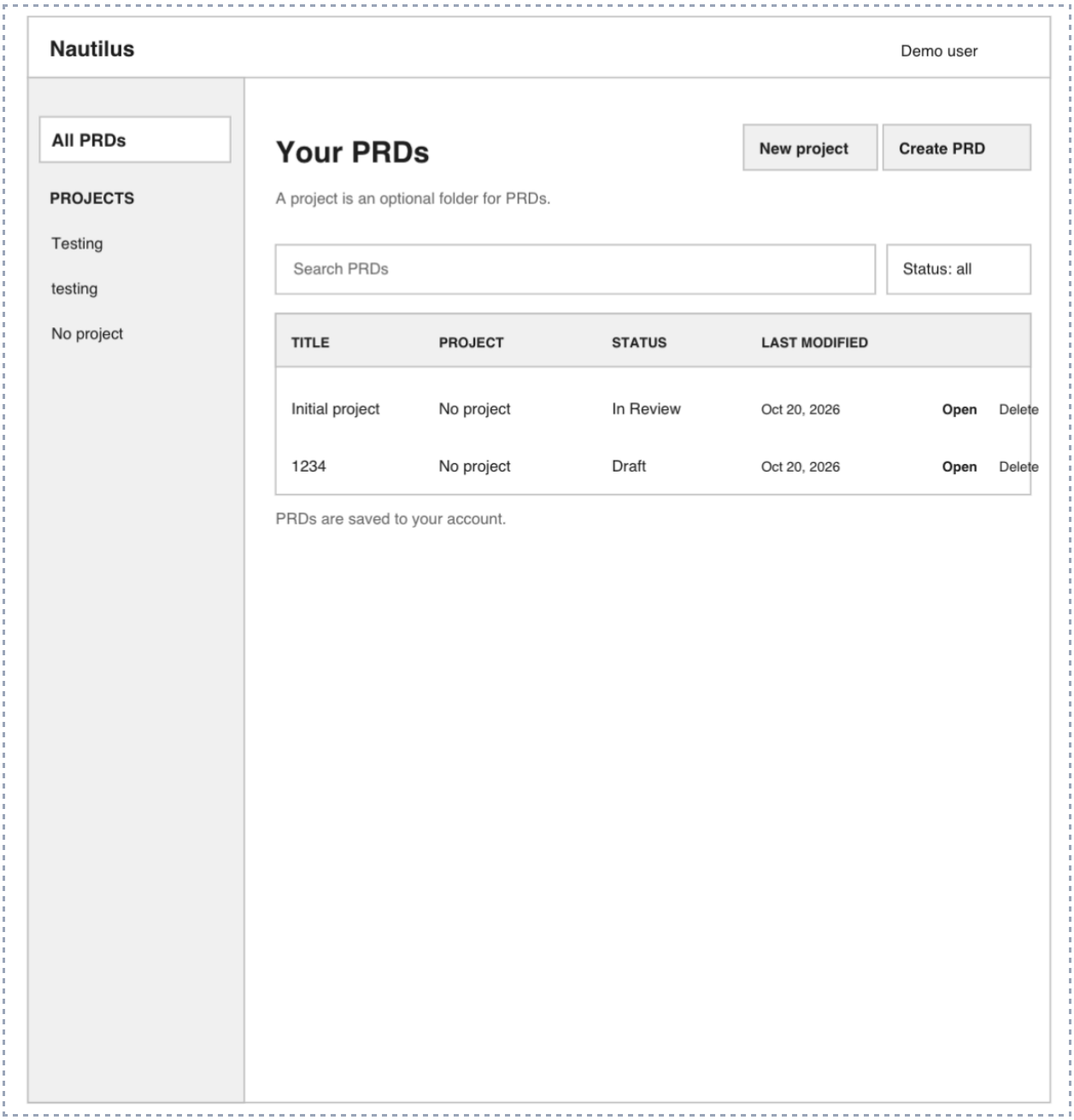
3.5 Edit / Regenerate Section

Screens	PRD Editor → Select Section → Edit or Regenerate
User story	US3, US4
Requirements	R_5, R_6
Description	In the PRD editor, the user selects and edits content, then selects Save. The AI assistant displays section-specific actions, but its chat response is explicitly a prototype placeholder until the AI is connected.



3.6 Save / Reopen PRD

Screens	Dashboard → Existing PRD → Editor
User story	US11
Requirements	R_7
Description	The dashboard lists saved PRDs with title, project, status, and last-modified date. The user selects Open to return to the document editor and continue work.



3.7 UX Principles Review

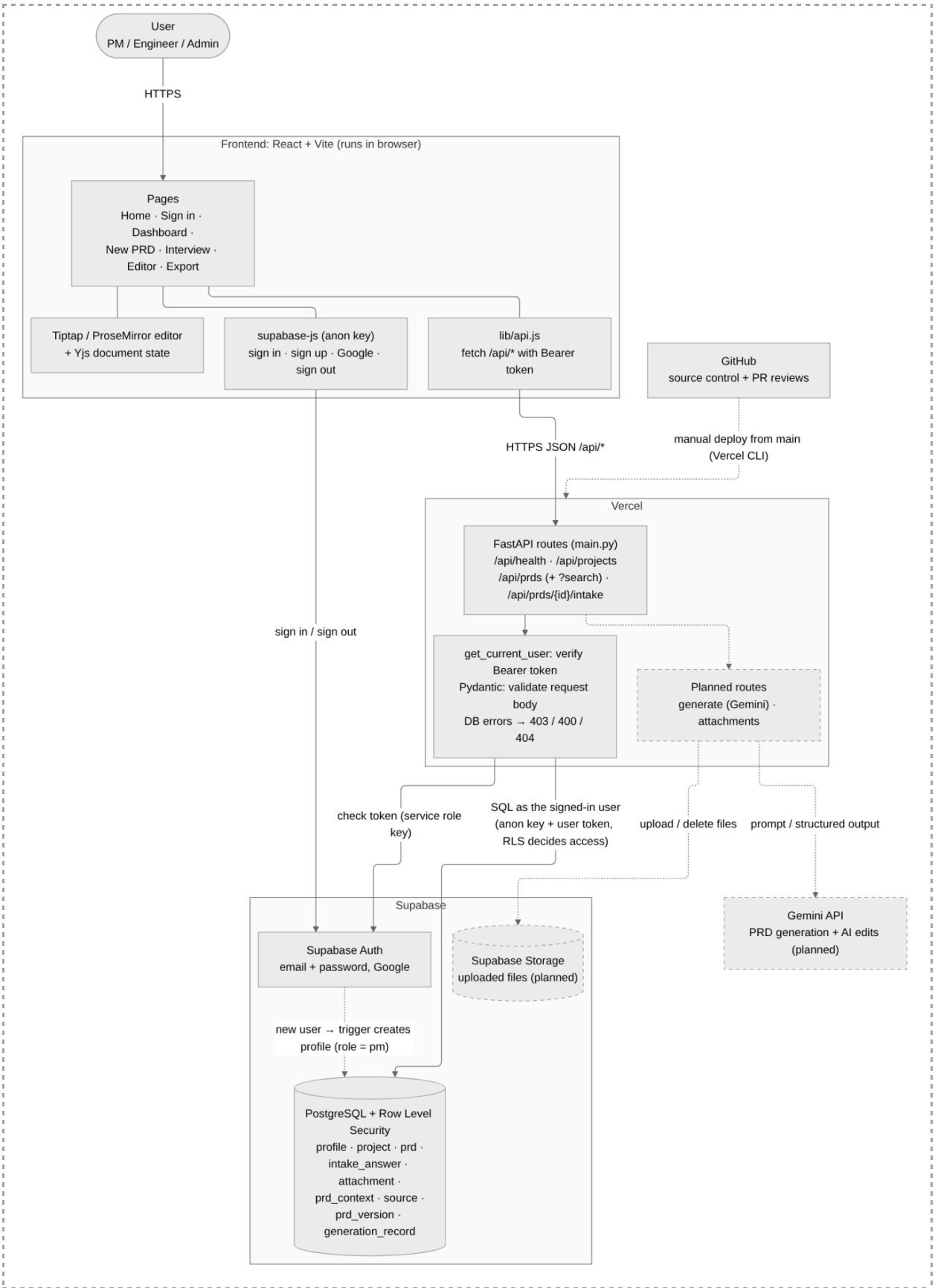
Principle	Question	How our design meets it
Useful	Does it actually help the user?	Guided questions turn a product idea into PRD input, followed by a generated structured draft
Usable	Can someone figure it out easily?	A numbered question sidebar, required-field labels, Next, Back, and review/edit controls support an understandable sequence.
Desirable	Does it feel worth using?	The dashboard, step-by-step interview, and ready confirmation make the path from idea to editable PRD clear
Accessible	Can more people use it?	Inputs have visible labels, required fields are identified, and the active field has a visible focus outline.
Findable	Can users find what they need?	The dashboard includes Search PRDs, Status filter, project sidebar, and Open controls; the editor has a section outline.
Credible	Can the user trust it?	An assumption is explicitly flagged for review; the editor exposes Save and version history.

4. High-Level Architecture and Database Organization

Owner: Marco (lead), Vineela **Support:** Om (AI architecture), Philip (deployment flow)

Implemented in app/backend/schema.sql, matching Section 1 exactly. Verified by running the complete script against a throwaway Postgres instance with Supabase's auth.users/auth.uid() stubbed, then smoke-testing real inserts and cascade deletes across the full chain (profile → project → prd → prd_version → intake_answer) before applying it to the live Supabase project.

4.1 High-Level System Architecture


User
PM / Engineer / Admin
HTTPS
Frontend: React + Vite (runs in browser)
Pages
Home · Sign in · Dashboard · New PRD · Interview · Editor · Export
Tiptap / ProseMirror editor + Yjs document state
supabase-js (anon key)
sign in · sign up · Google · sign out
lib/api.js
fetch /api/* with Bearer token
GitHub
source control + PR reviews
HTTPS JSON /api/*
manual deploy from main (Vercel CLI)
Vercel
FastAPI routes (main.py)
/api/health · /api/projects
/api/prds (+ ?search) ·
/api/prds/{id}/intake
sign in / sign out
get_current_user: verify Bearer token
Pydantic: validate request body
DB errors → 403 / 400 / 404
Planned routes
generate (Gemini) · attachments
check token (service role key)
SQL as the signed-in user (anon key + user token, RLS decides access)
upload / delete files
prompt / structured output
Supabase
Supabase Auth
email + password, Google
Supabase Storage
uploaded files (planned)
Gemini API
PRD generation + AI edits (planned)
new user → trigger creates profile (role = pm)
PostgreSQL + Row Level Security
profile · project · prd · intake_answer · attachment · prd_context · source · prd_version · generation_record

Layer	Responsibilities
Frontend **React + Vite**	Login, PRD creation wizard, document editor, dashboard, uploads, exports, and a temporary browser-memory AI chat buffer. A Tiptap/Yjs rich-text editor is planned for a later milestone. Calls the FastAPI API.
Backend **Python + FastAPI**	Validates requests, enforces PM/engineer/admin authorization, manages application records and attachments, and exposes REST endpoints. Calls Gemini for PRD generation/edits, validates structured results, merges approved context patches, saves snapshots, and prepares exports.
Database, Auth, Files **Supabase**	PostgreSQL stores profile, project, prd, intake_answer, attachment, prd_context, source, prd_version, and generation_record. Supabase Auth owns sign-in/session credentials; profile.id matches its user UUID. Supabase Storage holds uploaded files.
AI **Gemini**	Receives prd_context, current document text, the temporary session chat buffer, and the new user request; no conversation history is retained by Gemini between calls.
Deployment **Vercel**	Live at csc648-team05.vercel.app. The Vite frontend and FastAPI backend deploy as one Vercel project, with the backend as Python serverless functions. Deploys are currently manual through the Vercel CLI.

Primary data path: Browser (React) → FastAPI → Supabase PostgreSQL → FastAPI → Browser.

AI path: Browser → FastAPI → Gemini → validated document/context updates → PostgreSQL → Browser.

4.2 Database Organization

These field names and limits follow the team's Data Definitions V2.

Table	Main fields / constraints	Purpose
profile	id (Auth UUID), full_name (≤100), email (unique), role (pm | engineer | admin), team (≤100), created_at, updated_at	User metadata and access role; passwords are handled by Supabase Auth.
project	id, name (≤200), owner_id → profile.id, created_at, updated_at	Folder for PRDs only; no independent AI context.
prd	id, title (≤200), owner_id, project_id (nullable), status, current_version_id, content_json, content_text, ydoc_state (nullable), doc_schema_version, revision, created_at, updated_at	Primary document; standalone PRDs allowed. ydoc_state is canonical when present; JSON/text are derived by the browser.
intake_answer	id, prd_id, question_key, answer_text (≤5,000), skipped, answered_at; unique (prd_id, question_key)	Saves each wizard answer immediately; questions are fixed in application code.
attachment	id, prd_id, file_name, file_type, size_bytes, storage_path, uploaded_at	Metadata for up to 5 files per PRD, 5 MB each (.txt/.md/.pdf/.docx).

Table	Main fields / constraints	Purpose
prd_context	id, prd_id (unique), context_json, running_summary (≈4,000 chars), version, updated_at, updated_by	Per-PRD durable AI decisions, constraints, open questions, sources and summary.
source	id, prd_id, url, title, publisher, excerpt, retrieved_at, verification_status, added_by	Source provenance and verification status.
prd_version	id, prd_id, content_json, doc_schema_version, created_by, origin, created_at	Snapshots for undo/history; snapshot before AI edits.
generation_record	id, prd_id, trigger, model, prompt_version, context_version, grounding_used, status, error_message, user_request, created_at	Records generation activity and failures; not chat memory.

Future-only (priority 3): classification_suggestion for Phase 2 routing/approval recommendations; comment for Phase 3 collaboration. They are outside the current vertical-prototype scope.

Document structure: section types problem, goals, non_goals, target_users, user_stories, requirements, success_metrics, risks, and open_questions. Allowed Tiptap nodes and marks must be validated before accepting AI content.

4.3 Add / Delete / Search Architecture and Access Control

Table	Add / search / update / delete / display
profile	Create after Auth registration; find by id/email; update permitted profile fields; admin-managed deletion; display permitted profile information.
project	Create folder; list/search by owner and name; rename; delete with count confirmation and cascade; display accessible folders.
prd	Create standalone or inside project; search title/content_text and filter by project; update document state and metadata; delete with authorization; display accessible PRDs.
intake_answer	Insert on question submission; look up by prd_id + question_key; revise answer; cascade delete; prefill wizard.
attachment	Upload file and metadata; list by prd_id; replace if supported; delete file plus metadata; display attachment details.
prd_context	Initialize one per PRD; retrieve by prd_id; validate/merge AI patches and increment version; cascade delete; expose relevant decisions/questions.
source	Add URLs/citations; retrieve by PRD; update verification; cascade delete; display verified/unverified state.
prd_version	Create snapshots; retrieve history; delete with PRD; show previous versions/undo if implemented.
generation_record	Insert generation attempt; filter by PRD/status; update success/failure; cascade delete; internal diagnostics only.

Authorization: Authorization is enforced via Postgres Row Level Security, not just application code. PMs and admins create/edit/delete their own projects and PRDs; admins manage everything.

Engineer access uses a status rule rather than a persisted sharing table — engineers can view PRDs with status final or assigned, mirrored to that PRD's child records (intake answers, attachments, context, sources, versions, generation log). This was chosen over a prd_access table per the team's decision to defer dedicated sharing until it's actually needed. Role lookups use a SECURITY DEFINER helper function to avoid RLS recursion on the profile table itself.

Cascade behavior: deleting a project deletes its PRDs; deleting a PRD cascades to intake_answer, attachment, prd_context, source, prd_version, generation_record and later-phase dependent records. Supabase Storage objects require a separate backend cleanup operation.

4.4 Major Backend API Endpoints

Workflow	Route	Status	Behavior
Sign up/ Sign in/ Log out	No backend route	Built	The browser signs up, signs in, and signs out directly with Supabase Auth
Create Project	POST /api/projects	Built	Create a new project folder owned by the authenticated user.
Rename Project	PUT /api/projects/{project_id}	Built	Update a project's name; checks ownership/role.
Delete Project	DELETE /api/projects/{project_id}	Built	Check ownership/role; cascade-deletes the project's PRDs and their dependent records.
Create PRD	POST /api/prds	Built	Create a standalone or project-associated draft.
Guided intake	PUT /api/prds/{prd_id}/intake /{question_key}	Built	Save or revise one wizard answer.
Generate PRD	POST /api/prds/{prd_id}/generate	Planned	Request Gemini output from intake/context; validate, save and log. The M2 prototype fakes generation in the browser, then saves the result with the real Create/Edit routes.
Retrieve / list	GET /api/prds GET /api/prds/{prd_id} GET /api/projects	Built	List/search authorized PRDs and projects; load document state.
Edit PRD	PUT /api/prds/{prd_id}	Built	Save the PRD’s content. Planned with the Tiptap editor: storing ydoc_state and checking content_json against the allowed nodes and marks.
AI edit	POST /api/prds/{prd_id}/ai-edit	Planned	Use persistent context + chat buffer; snapshot before applying changes.

Workflow	Route	Status	Behavior
List Versions	GET /api/prds/{prd_id}/versions	Planned	List saved snapshots for a PRD, newest first, for history/undo.
Restore Version	POST /api/prds/{prd_id}/versions/{version_id}/restore	Planned	Load a snapshot's content_json back into the PRD, record a new checkpoint version, and update current_version_id.
Delete PRD	DELETE /api/prds/{prd_id}	Built	Check ownership/role; remove dependent data and Storage files.
Upload file	POST /api/prds/{prd_id}/attachments	Planned	Validate extension, size and count; upload file and save attachment metadata.
Export PRD	No backend route	Built (In Browser)	PDF (through the browser's print / save as PDF), Word (.doc) and Markdown, all from the export screen

Built routes are live on the deployed site. All built routes require login, and the database's Row Level Security controls who can see or edit each PRD. Planned routes are designed but will be built after M2.

Restore Version creates a new checkpoint version rather than overwriting history, matching Section 1.9 ("Undo restores a version by loading its content_json back into the editor"), so undo itself is undoable.

For an uploaded file to become a new editable PRD (rather than an attachment to an existing PRD), the team must separately define a parsing/import endpoint and conversion rules.

4.5 External APIs, Frameworks, and Open-Source Components

Component	Use
React + Vite	Browser interface and build tool
Tiptap/ProseMirror + Yjs	Planned rich-text editor
Python + FastAPI	REST API, application logic and validation
Supabase PostgreSQL, Auth and Storage	Data, authentication and uploaded files
Gemini API	Structured PRD drafting, edits and context patches
Document import/export libraries	To be chosen and documented once formats are implemented
Vercel	Hosting (live at csc648-team05.vercel.app)
GitHub	Source control and review

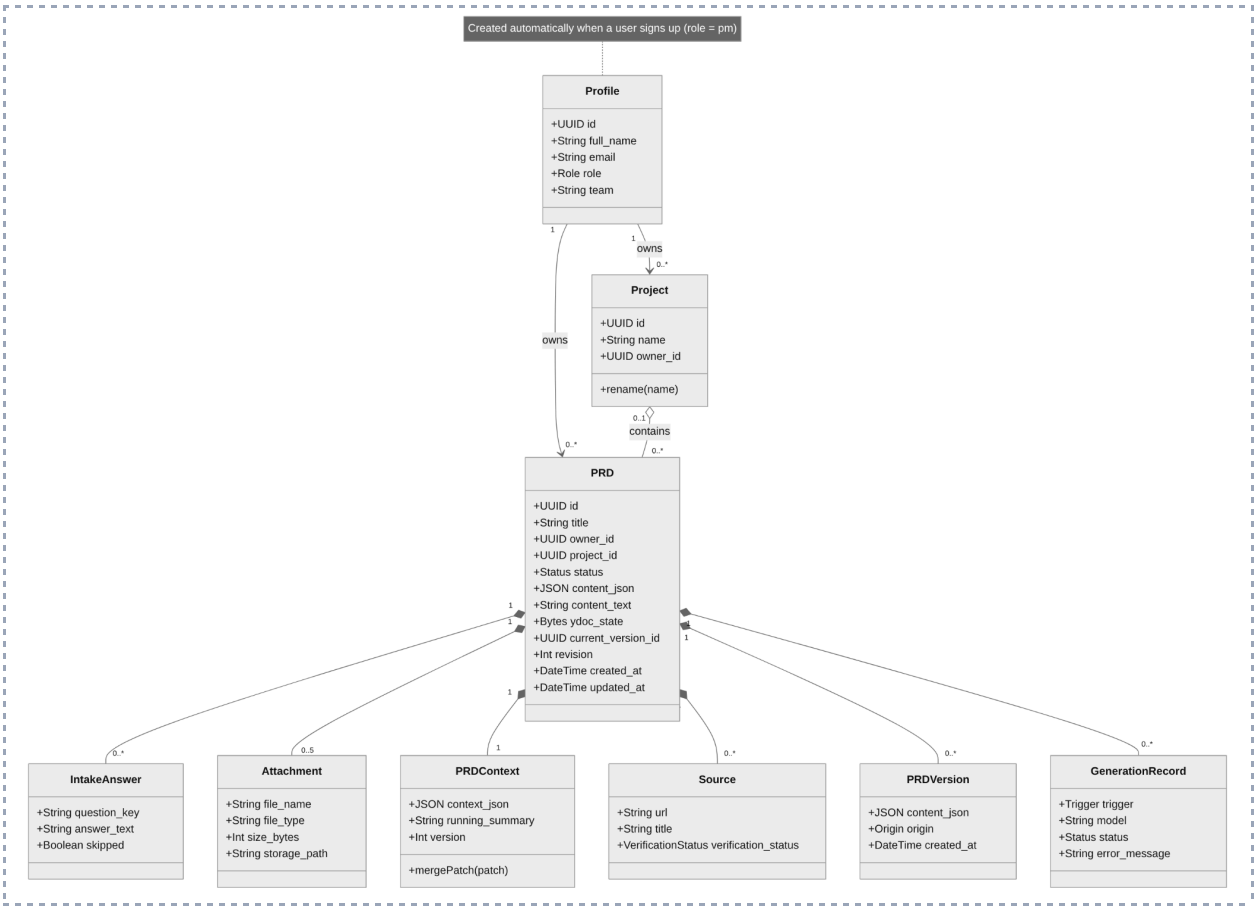
4.6 Changes to Tools and Frameworks Since M1

1. AI provider chosen: Gemini (was TBD in M1).
2. Added Supabase Storage for uploaded files. Planned: Tiptap/ProseMirror + Yjs for the PRD editor.
3. Schema from Section 1 (Data Definitions V2) is now implemented in Supabase, not just proposed — profiles/prds (the M0 placeholder schema) have been replaced with the full nine-table schema.

5. High-Level UML Diagrams

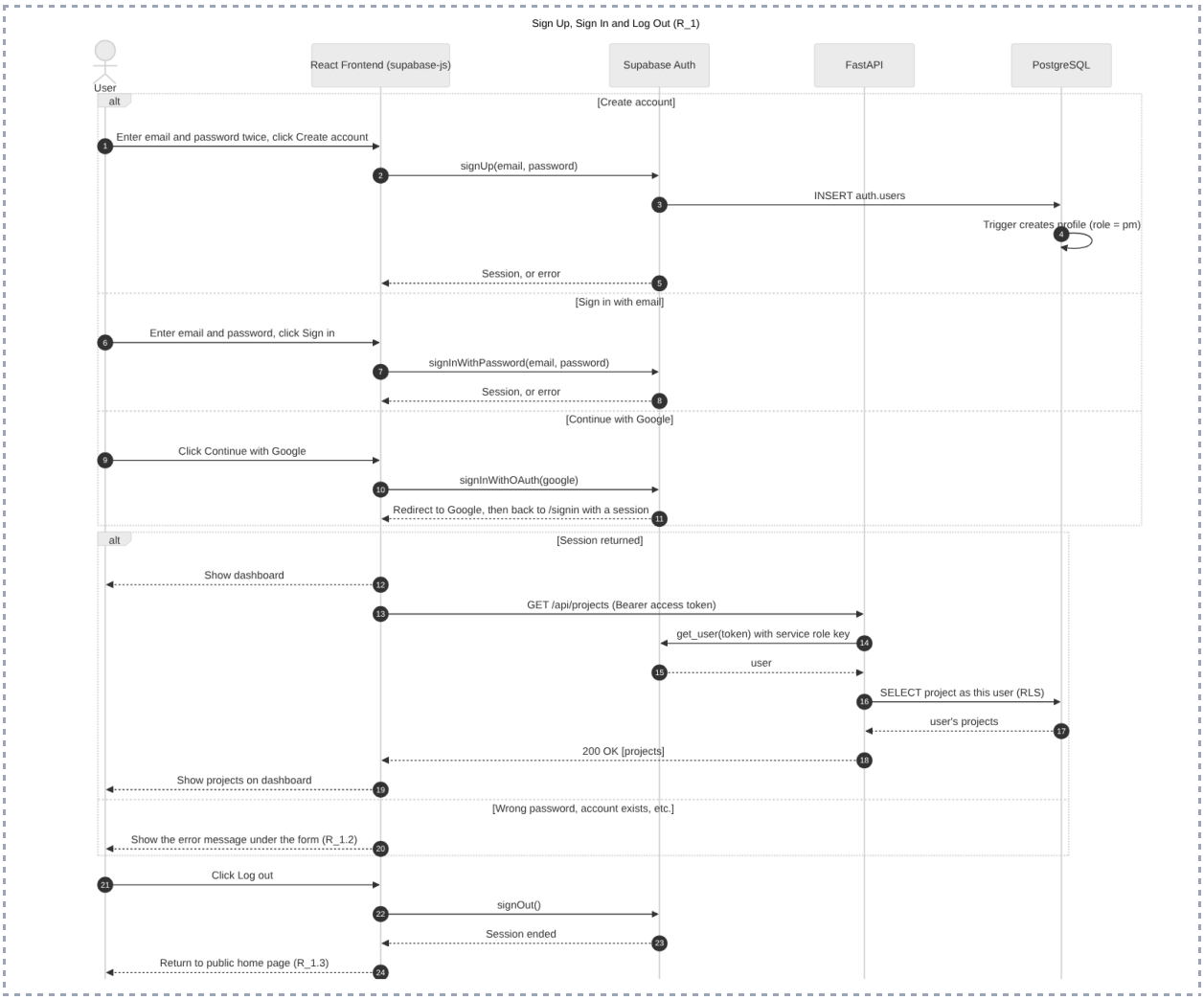
Owner: Marco, Vineela **Support:** Om , Gio (diagram) (AI sequence), Aksh (frontend flow)

5.1 Class Diagram

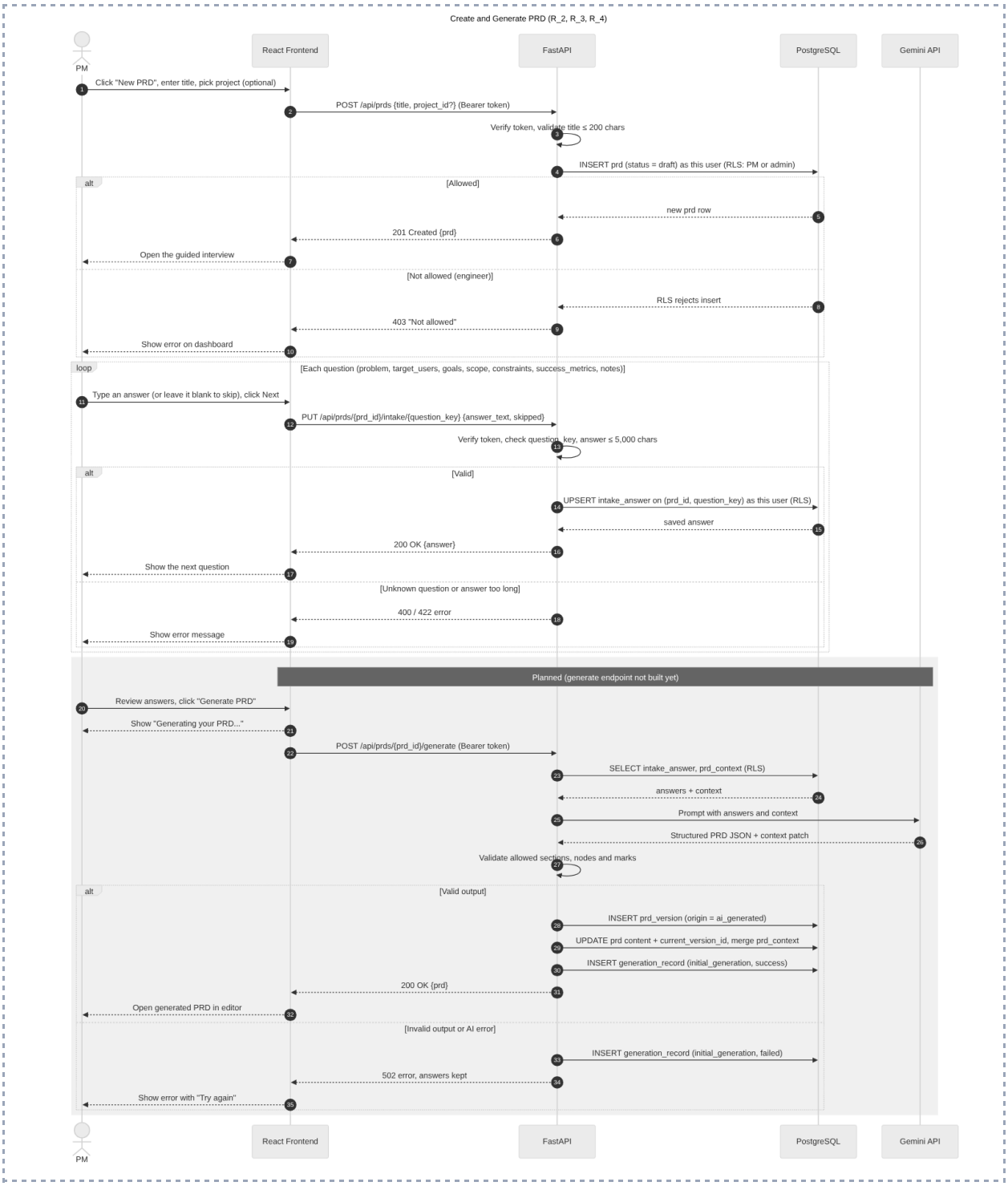


5.2 Sequence Diagrams

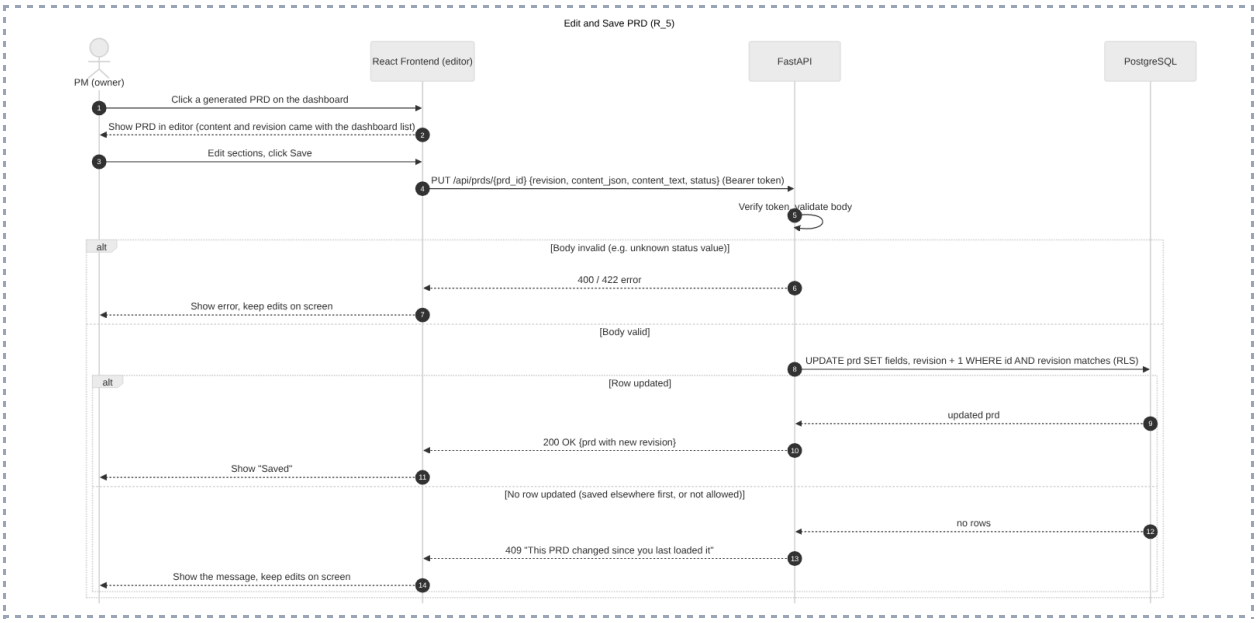
5.2.1 Sign Up, Sign In, and Log Out (R_1)



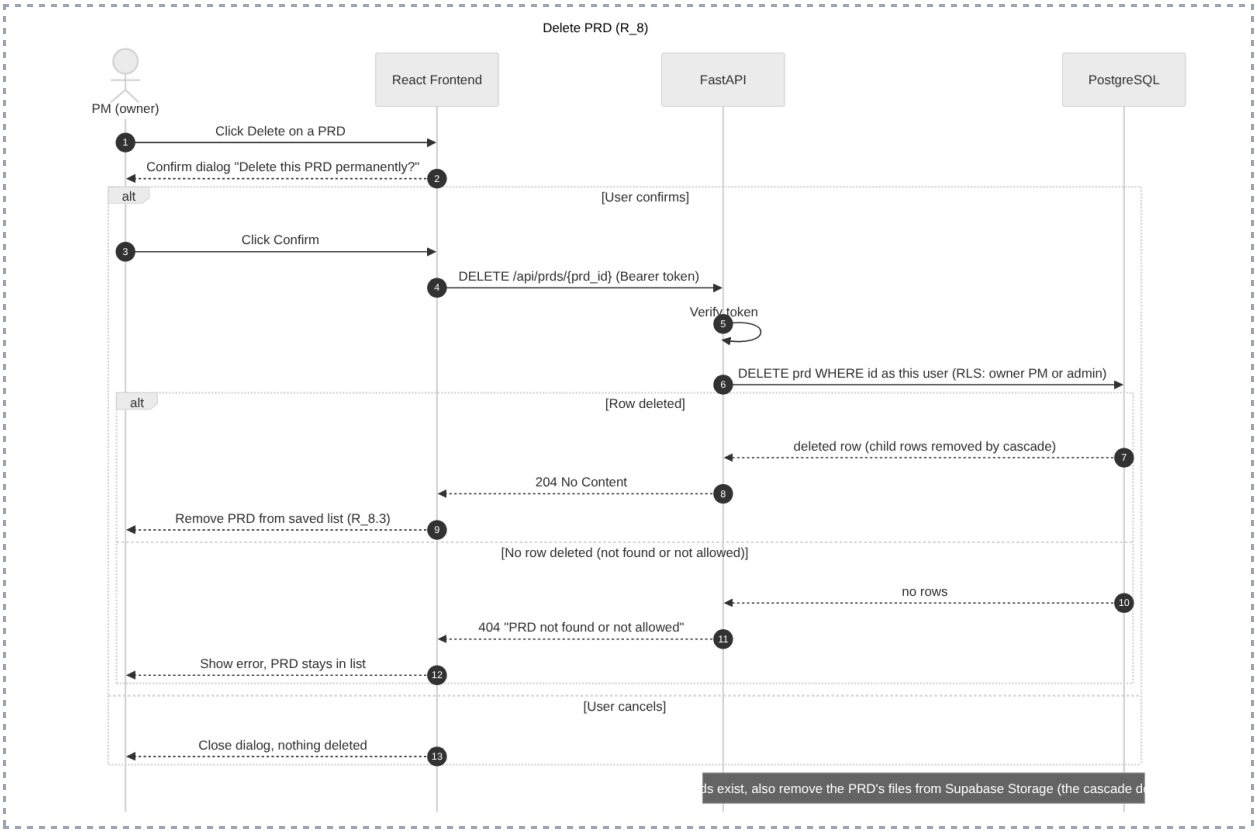
5.2.2 Create and Generate PRD (R_2, R_3, R_4)



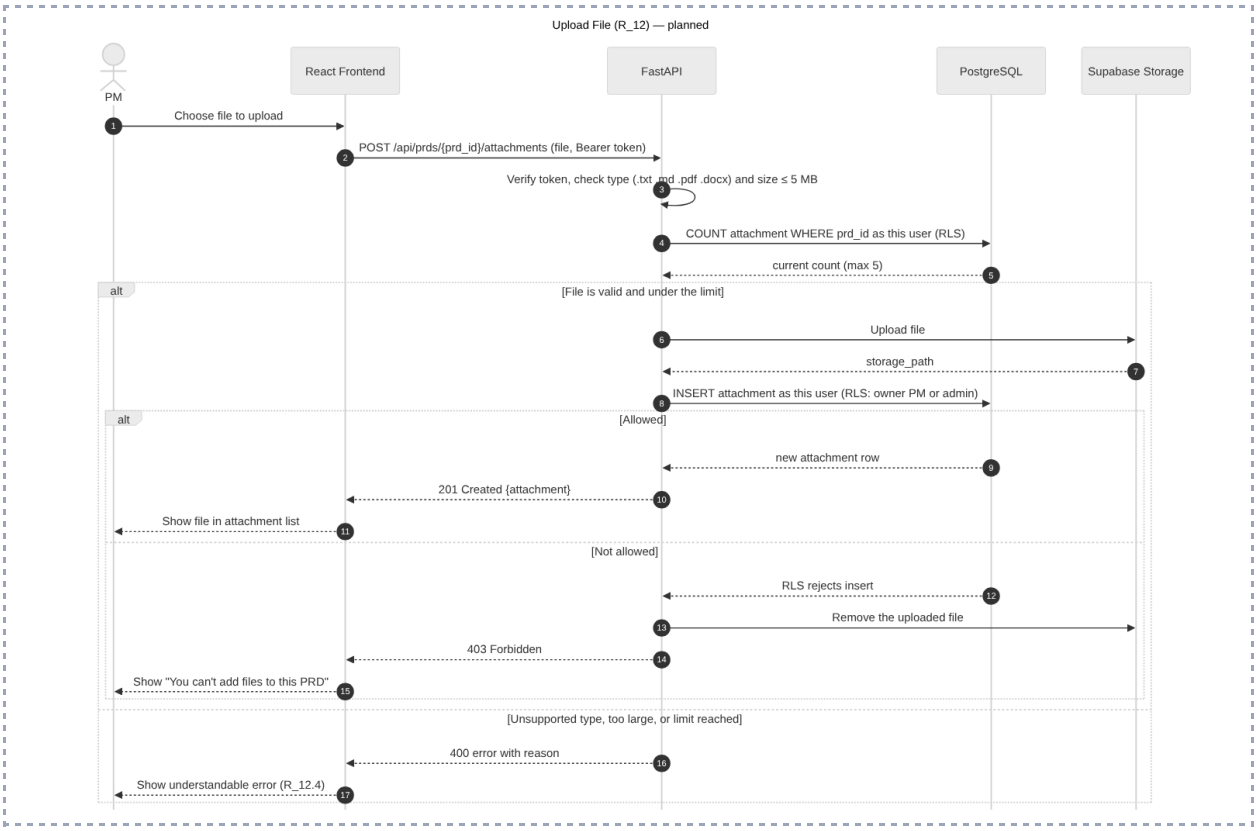
5.2.3 Edit and Save PRD (R_5)



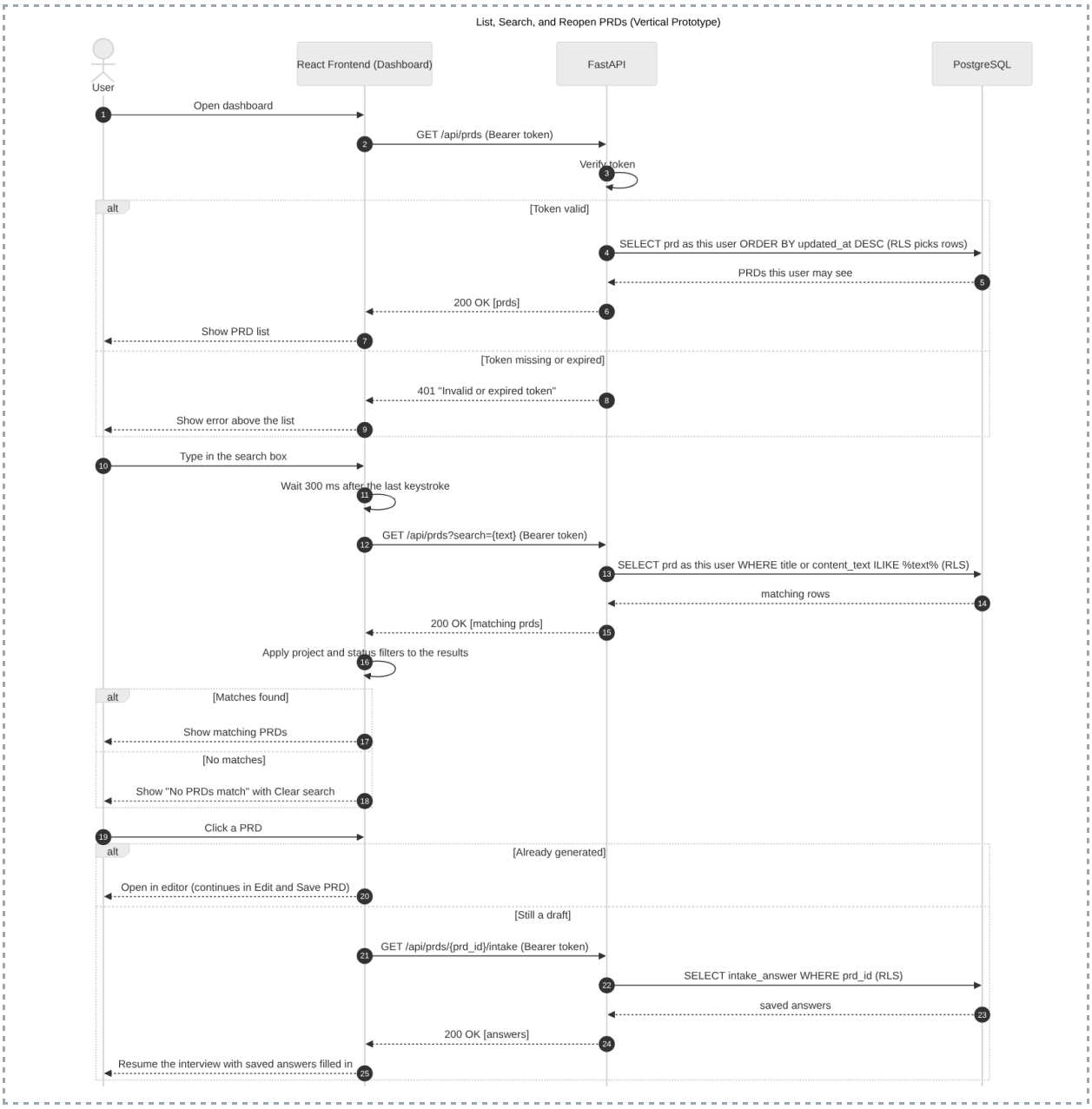
5.2.4 Delete PRD (R_8)



5.2.5 Upload File (R_12)



5.2.6 List, Search, and Reopen PRDs (Vertical Prototype)


List, Search, and Reopen PRDs (Vertical Prototype)
User
React Frontend (Dashboard)
FastAPI
PostgreSQL
1 Open dashboard
2 GET /api/prds (Bearer token)
3 Verify token
alt
[Token valid]
4 SELECT prd as this user ORDER BY updated_at DESC (RLS picks rows)
5 PRDs this user may see
6 200 OK [prds]
7 Show PRD list
[Token missing or expired]
8 401 "Invalid or expired token"
9 Show error above the list
10 Type in the search box
11 Wait 300 ms after the last keystroke
12 GET /api/prds?search={text} (Bearer token)
13 SELECT prd as this user WHERE title or content_text ILIKE %text% (RLS)
14 matching rows
15 200 OK [matching prds]
16 Apply project and status filters to the results
alt
[Matches found]
17 Show matching PRDs
[No matches]
18 Show "No PRDs match" with Clear search
19 Click a PRD
alt
[Already generated]
20 Open in editor (continues in Edit and Save PRD)
[Still a draft]
21 GET /api/prds/{prd_id}/intake (Bearer token)
22 SELECT intake_answer WHERE prd_id (RLS)
23 saved answers
24 200 OK [answers]
25 Resume the interview with saved answers filled in

6. Actual Key Risks

Owner: Om, Leman **Support:** All members provide real risk

Category	Risk	How We Will Resolve It	Owner
Skills	Our project uses React, FastAPI, Supabase, and Gemini. Learning unfamiliar parts and connecting them may take extra time.	We will study the tools needed for our tasks, share examples and setup notes, and help each other with difficult parts.	Aksh, Marco, Vineela, Om
Schedule	Saving and reopening a PRD needs both frontend and backend work. If one part takes longer, related tasks and testing may be delayed.	We will set dates for connected tasks and check progress during meetings. We will share schedule changes on Discord so everyone can adjust their work.	Om, Leman
Legal / Content	We use open-source libraries and sample documents. Their licenses may require credits or limit how we can use them.	We will check library licenses and include required credits. For testing, we will use our own documents or files we have permission to use.	Philip, Marco, Vineela
Teamwork	Some tasks depend on another member's work. For example, the frontend needs to know which data the backend expects and returns. A change on one side can affect the other side.	We will agree on the API details together and share changes on Discord before others continue their work. If a task is blocked, we will discuss it early so teammates can help.	Aksh, Marco, Vineela
Technical	The prototype may work on our computers but fail after deployment. Database connections or long AI requests could prevent PRDs from loading or generating.	We will test database listing, search, and AI requests on the deployed site as we build it, leaving time to fix problems before the demo.	Philip, Marco, Om

7. Project Management

Owner: Leman (lead), Om **Support:** Philip (Git/process details)

7.1 Meetings and Progress Sharing

We meet in person twice a week, on Thursdays at San Mateo Library and during class. At the beginning of each milestone, our Team Lead assigns tasks, so everyone knows what they need to work on. During meetings, we share what we have finished, what we are still working on, and where we need help. We also review our work together and discuss ideas. If someone gets stuck, they can ask during a meeting or on Discord. Everyone is willing to help each other.

After each meeting, our Scrum Master posts a summary in the Discord meeting channel. It includes what we worked on, the designs we discussed, and the decisions we made. Anyone who could not attend or missed a detail can read the summary and catch up. Between meetings, we all actively use Discord to share updates and ask questions, so everyone can follow the team's progress.

7.2 Task Tracking

We keep our task assignments in both Discord messages and the milestone PDF. Each task has a team member’s name next to it, so we can check who is working on which part. We usually plan to finish our tasks before the next meeting, but we discuss the timing together depending on how much work each task needs.

At our meetings, we tell each other whether we have finished our tasks or are still working on them. If something is taking longer than expected, we talk about what is left and whether anyone can help. We also use Discord between meetings to ask questions and keep each other updated.

7.3 Code Workflow

Team members who work on the code create their own branches for their assigned tasks, named feature/[name]-[description]. When their work is ready, they open a pull request into main. At least one other teammate must review and approve it before the GitHub Master merges it. This allows members to work on their own tasks while bringing everyone’s contributions together in the same project.

8. Response to Professor Feedback and Planned Enhancements

The feedback from our M2 demo highlighted the need to explain what Nautilus offers beyond generating a PRD with AI. General AI tools can already create documents, so Nautilus needs to support more of the work involved in managing product requirements. We also need to identify the Priority 1 features we can realistically complete by the end of the semester.

The feedback focused on how a PRD moves between departments, who is responsible for reviewing and approving it, and how changes are shared with the people involved. For example, Finance may reduce the budget, or Engineering may identify a technical limitation. Teams need to see how these changes affect the requirements and what needs to be reviewed again.

After the demo, we met to discuss these points. We agreed to expand Nautilus into a shared workspace where teams can create, review, edit, and approve PRDs. We will build on our current architecture and add support for organizations, permissions, and the review process.

8.2 Planned Features

The following features reflect the direction we agreed on during our meeting. We will confirm the final P1 scope, detailed permissions, and implementation order as a team.

8.2 Planned Features

After discussing the demo feedback, we agreed on the following additions to Nautilus. These features will help teams work on a PRD together, review it, approve it, and follow later changes. We will build on the current application and its revision tracking.

Feature	What We Plan to Add	Purpose
PRD and Project Details	We will add a detail page showing the PRD's description, related project, owner, assigned members, current status, and last update. Users will also be able to check approval progress and revision history from this page. This will help them understand where the PRD stands without opening the editor or asking teammates for an update.	Keep the PRD's status, ownership, and review information in one place.
Organization and Team Management	Admins will be able to add employees to an organization and arrange them into departments and teams, such as Product, Engineering, Finance, and Marketing. A team directory or organization view will show how these groups are organized. We discussed invitations and admin-managed accounts; the exact method for adding employees still needs to be decided.	Support collaboration between departments within the same organization.

Roles and Permissions	A person's department and their permissions will be managed separately. For example, two people in Marketing may have different access: one can edit a PRD, while the other can only view it. Admins will manage access, and only authorized users will be able to edit, delete, or approve PRDs. We will agree on the final role names and permission rules before implementation.	Make responsibilities clear and control who can change or approve a PRD.
Assigned Members and Reviewers	Authorized users will be able to assign employees or teams to work on or review a PRD. These assignments will appear in the detail page so everyone can see who is involved. Assignment will show who is expected to contribute, while revision history will show who actually made changes.	Show who is expected to work on the PRD and who needs to review it.
Comments and Cross-Departme nt Feedback	Members will be able to comment on the whole PRD or a specific section. Comments will show the author and department. Engineering could flag a technical limitation, while Finance could raise a budget concern. Reviewers requesting changes will be able to explain what needs to be addressed before they approve the document.	Keep each department's concerns connected to the relevant requirements.
Approval Workflow and Progress	PRDs will be sent to designated approvers, who can approve them or request changes with an explanation. An approval panel will show completed and pending reviews and the responsible person or department. For example, Finance may have approved a PRD while Engineering is still reviewing it. The progress shown here will track reviews and approvals, not software development.	Give owners and reviewers a clear view of completed approvals and pending reviews
Revision History and Reapproval	We will extend the existing version history to show who changed a PRD, what changed, and when. If an approved PRD is edited, the updated version will need approval again. Previous versions will remain available so users can check the earlier content. We will define which reviewers need to approve an updated version as part of the approval rules.	Make changes traceable and ensure approval applies to the version that was reviewed.
Notification Center	An in-app notification center will show relevant assignments, review requests, comments, requested changes, and approval decisions. Each notification will link to the related PRD. Users will also have "Needs My Approval" and "Waiting for Approval" views. We will decide who receives each type of update rather than notifying everyone about every edit. Email notifications remain an optional addition.	Help users notice tasks and review decisions that need their attention.

AI Support for Review Feedback	The PRD owner will be able to ask AI for possible solutions or revisions based on feedback raised during review. For example, if Finance rejects the proposed budget, AI could suggest reducing the scope or dividing the work into phases. Team members will review these suggestions before applying changes. AI will not approve the document or make the final decision.	Help the team respond to review concerns while keeping decisions with people.

8.3 Example Review Workflow

A product manager creates a PRD and assigns Engineering and Finance members to review it. Both departments can read the document and leave feedback. Their review progress appears in the approval panel.

If Finance requests changes because the proposed work exceeds the budget, the product manager can review the comment and ask AI for possible ways to adjust the requirements. The product manager then decides what to change and saves the revised document. The revision history records the update, and the PRD goes through review again.

Once the required approvals are complete, the PRD is marked as approved. If its content changes later, the updated version will need approval again. Users can follow the document’s status through the detail page, approval panel, and notifications.

These additions will build on our existing PRD creation and editing workflow. We will keep the current visual style and add screens for team management, review, and notifications. We will also update the related requirements, data definitions, and diagrams to match the agreed scope.